

Trabalho Prático: plataforma de aprendizagem

Alexsandro Santos Soares

prof.asoares@ufu.br

Universidade Federal de Uberlândia

Faculdade de Computação

Informações importantes

- **valor:** 35 pontos
- **Data de entrega e apresentação:** 18/09/2025 (quinta-feira)
- **Grupo:** 10 membros
 - enviar impreterivelmente até o dia [08/09](#), segunda-feira, os nomes dos membros pelo Teams.
- **Modo de envio:** link para o GitHub
- **LLMs:** as regras para o uso de LLM's são as mesmas do primeiro trabalho.

1 Introdução

As técnicas de repetição espaçada, recordação ativa e método Feynman são estratégias cientificamente comprovadas para potencializar o aprendizado e a retenção de informações.

A repetição espaçada consiste em revisar conteúdos em intervalos crescentes de tempo, fortalecendo a memória de longo prazo e reduzindo o esquecimento.

A recordação ativa, por sua vez, exige que o estudante recupere voluntariamente o conhecimento, seja respondendo perguntas, praticando com flash-cards ou testando-se sem consultar o material, o que aprofunda a fixação e revela lacunas de entendimento.

Já o método Feynman propõe que o estudante explique conceitos em linguagem clara e simples, simulando o ensino para uma criança, o que incentiva o domínio real do assunto e a identificação de pontos frágeis na compreensão.

As técnicas acima servem para tornar o estudo mais eficiente e garantir que o aprendizado seja duradouro, significativo e aplicável em diversos contextos acadêmicos ou profissionais.

Repetição Espaçada

A **repetição espaçada** é uma técnica de aprendizagem baseada em princípios sólidos da neurociência e da psicologia cognitiva. O método parte da constatação experimental de que revisar informações em intervalos crescentes após o primeiro contato leva a uma retenção muito mais duradoura dos conteúdos, reduzindo o fenômeno do esquecimento entre revisões.

O embasamento científico da repetição espaçada advém de pesquisas, como as de Hermann Ebbinghaus (século XIX), que mostraram de forma empírica a evolução da *curva do esquecimento*. Com base nisso, algoritmos contemporâneos, como o SM-2 (utilizado em aplicativos como Anki), foram desenvolvidos para programar revisões otimizadas conforme o desempenho do estudante, tornando possível reter até 95% do conteúdo após múltiplos ciclos espaçados, especialmente quando a revisão respeita o princípio de “testes de recuperação”.

A repetição espaçada é ainda mais eficaz quando associada a **recordação ativa**, pois estimula não apenas a revisitação passiva do material, mas a recuperação ativa da informação por meio de perguntas, testes e flashcards¹. Diversos estudos mostram que a tentativa de recordar de memória fortalece as conexões neurais, tornando a rememoração mais eficiente e duradoura.

Recordação Ativa

A **recordação ativa** é a prática de recuperar informações voluntariamente, ao invés de apenas ler ou sublinhar conteúdos. Exemplos incluem tentar responder perguntas sem consultar as respostas, resumir ideias em uma folha em branco, ou explicar conceitos complexos sem auxílio de fontes externas.

Trata-se de um método fundamentado em pesquisas recentes, por exemplo, de Henry Roediger e Jeffrey Karpicke (Universidade de Washington), que demonstram que o efeito da *testagem* – a realização de testes de recordação –

¹Veja a revisão do tema: <https://quizlet.com/content/science-behind-space-d-repetition>

é superior, tanto em retenção a curto prazo quanto para aprendizagem profunda². A explicação científica reside no fato de que a recuperação voluntária aumenta a força e a acessibilidade das representações de memória.

Além de potencializar o aprendizado, a recordação ativa permite ao estudante identificar lacunas de conhecimento e corrigir equívocos antes que se consolidem.

²Veja: <https://exame.com/carreira/guia-de-carreira/5-formas-de-aplicar-a-recordacao-ativa-para-estudar-melhor/>

Método Feynman

O **Método Feynman**, criado pelo físico e Nobel Richard Feynman, integra princípios de simplificação, ensino e revisão como estratégia central para aprofundar o entendimento de qualquer conteúdo³.

O método tem quatro etapas fundamentais:

1. **Escolha do conceito:** selecionar o conteúdo ou tópico a ser aprendido e estudar o material inicial.
2. **Explicação simplificada:** tentar explicar o conceito como se estivesse ensinando uma criança, evitando jargões técnicos e recorrendo à linguagem comum.
3. **Identificação de lacunas e revisão:** retornar ao material para identificar onde a explicação falhou ou onde se encontram as lacunas do entendimento.
4. **Simplificação e analogias:** reorganizar as anotações, criar exemplos e analogias para facilitar a retenção, refinando a compreensão.

Esse método possui robusto embasamento científico porque incorpora *ensinar para aprender*, um dos métodos mais eficientes segundo a Pirâmide de Aprendizagem de William Glasser, que afirma que ensinamos 90% do que aprendemos. O processo da técnica Feynman força a mente a criar conexões, simplificar tópicos e identificar falhas de compreensão, elementos centrais para a neurociência do aprendizado.

Estudos mostram que **a habilidade de ensinar está diretamente relacionada à proficiência no entendimento do conceito**, e que a incorporação de analogias, linguagem simples e revisão ativa favorece o armazenamento em memória de longo prazo.

O método se destaca especialmente nos ambientes universitários e preparatórios, pois promove autonomia, autoavaliação e capacidade de síntese, além de preparar o estudante para situações de exposição e avaliação oral, como provas discursivas, entrevistas ou apresentações.

Conclusão

Repetição espaçada, recordação ativa e o método Feynman representam técnicas de aprendizagem eficazes e cientificamente validadas. Quando utilizadas

³Veja: <https://exame.com/carreira/guia-de-carreira/como-usar-a-tecnica-feynman-para-estudar-e-aprender-materias-dificeis/>

em conjunto, formam uma abordagem completa, que não apenas facilita a retenção, mas proporciona compreensão profunda e aplicabilidade do conhecimento, preparando o estudante de maneira consistente para desafios acadêmicos e profissionais.

2 Visão Geral do Projeto

O projeto descrito neste documento representa uma evolução natural dos projetos anteriores da disciplina. A novidade vem do uso de técnicas de repetição espaçada e recordação ativa com recursos avançados de uma plataforma moderna de aprendizagem Android.

O projeto possui como requisitos fundamentais o uso de múltiplos tipos de flashcards, integração GPS e armazenamento híbrido. Também integra inteligência artificial, recursos sociais e uma arquitetura robusta baseada em nuvem.

O aplicativo serve como ponte entre a experiência educacional clássica e as demandas do desenvolvimento mobile profissional. O projeto enfatiza *código limpo*, padrões modernos e integração escalável, preservando a praticidade dos projetos anteriores. Cada membro contribui com até 30h no ciclo de três semanas, aproveitando LLMs para acelerar aprendizado e implementação.

Cada membro da equipe contribuirá em um ciclo estruturado de duas semanas, preservando a carga de trabalho equilibrada enquanto incorpora amplo suporte de LLM para acelerar o aprendizado e a implementação.

O produto final será um aplicativo Android totalmente funcional que demonstra tanto a continuidade pedagógica da disciplina quanto a evolução para práticas de desenvolvimento de nível profissional.

Requisitos Fundamentais

2.1 Sistema Avançado de Flashcards

O app criará quatro tipos de flashcards, aprimorados com criação multimídia e IA:

- **Frente e Verso:** flashcards tradicionais com suporte a texto rico, imagens e áudio.
- **Cloze/Omissão:** sistema inteligente de lacunas com múltiplas palavras omitidas e dicas contextuais

- **Digite a Resposta:** entrada de texto com verificação inteligente usando IA para validação semântica
- **Múltipla Escolha (Quiz):** sistema dinâmico de alternativas com distratores inteligentes gerados por IA

O sistema tradicional de flashcards é expandido com capacidades multimídia profissionais, mantendo a simplicidade de uso que caracteriza ferramentas educacionais eficazes. A implementação utiliza Jetpack Compose para interfaces responsivas e intuitivas, enquanto integra processamento de linguagem natural para validação inteligente de respostas em texto livre.

```
sealed class FlashcardType {
    data class FrenteVerso(val frente: RichContent, val verso: RichContent) : FlashcardType()
    data class Cloze(val texto: String, val lacunas: List<ClozeGap>) : FlashcardType()
    data class DigiteResposta(val pergunta: RichContent, val respostasValidas: List<String>) : FlashcardType()
    data class MultiplaEscolha(val pergunta: RichContent, val alternativas: List<Alternativa>) : FlashcardType()
}
```

2.2 Sistema GPS Inteligente para Contextualização Espacial

Usar o sistema de GPS para favoritar até 7 localizações diferentes, agora integrado a um sistema inteligente de recomendações que otimiza a experiência de aprendizagem baseada no contexto espacial do usuário.

Funcionalidades Aprimoradas:

- **Geofencing Inteligente:** detecção automática de entrada/saída das localizações favoritas
- **Recomendações Contextuais:** sugestão de tipos de flashcards adequados para cada localização
- **Analytics Espaciais:** relatórios de desempenho por localização com insights sobre ambientes mais produtivos
- **Rotação Inteligente:** algoritmo que evita repetição excessiva de conteúdo na mesma localização

Implementação Técnica:

```
@Entity(tableName = "favorite_locations")
data class FavoriteLocation(
    @PrimaryKey val id: String,
    val name: String,
    val latitude: Double,
```

```
val longitude: Double,  
val radius: Int = 100,  
val preferredCardTypes: List<FlashcardType>,  
val studySessionCount: Int = 0,  
val averagePerformance: Double = 0.0  
)
```

2.3 Repetição Espaçada Avançada com IA

O projeto mantém o sistema de repetição espaçada como funcionalidade central, expandindo o algoritmo tradicional SM-2 com capacidades adaptativas alimentadas por inteligência artificial. O sistema analisa padrões individuais de aprendizagem, contexto espacial e desempenho histórico para otimizar intervalos de revisão.

O algoritmo considera a localização atual do usuário para evitar repetições excessivas do mesmo conteúdo no mesmo ambiente, preservando o requisito original enquanto adiciona sofisticação através da análise de padrões comportamentais e ambientais.

Os recursos avançados presentes neste sistema são:

- **Predição de Dificuldade:** IA analisa conteúdo para estimar dificuldade inicial
- **Adaptação Contextual:** Ajuste de intervalos baseado em localização e horário
- **Análise Comportamental:** Identificação de padrões de esquecimento individuais
- **Otimização Temporal:** Sugestões de horários ideais para revisão baseadas em desempenho histórico

3 Assistente de Estudos Potencializado por IA: Especificação Técnica Detalhada

3.1 Arquitetura Inteligente de Assistência Educacional

O Assistente de Estudos Potencializado por IA constitui o diferencial central deste projeto, integrando múltiplas capacidades de inteligência artificial para criar uma experiência de aprendizagem verdadeiramente adaptativa

e personalizada. Este sistema vai além da simples geração de conteúdo, implementando uma arquitetura cognitiva que compreende, analisa e otimiza continuamente a experiência educacional individual.

3.2 Sistema Multi-LLM com Fallback Inteligente

Arquitetura de Orquestração: O sistema implementa uma arquitetura de múltiplas camadas com diferentes modelos especializados, garantindo disponibilidade contínua e otimização de custos através de estratégias inteligentes de roteamento de requisições.

Implementação Técnica:

```
sealed class LLMPProvider {
    data class Primary(val apiKey: String, val model: String) : LLMPProvider()
    data class Secondary(val endpoint: String, val headers: Map<String, String>) : LLMPProvider()
    data class LocalFallback(val modelPath: String) : LLMPProvider()
}

class IntelligentLLMOrchestrator {
    private val providers = listOf(
        LLMPProvider.Primary("google-ai-key", "gemini-2.5-flash"), // Custo otimizado
        LLMPProvider.Secondary("groq-endpoint", mapOf("Authorization" to "Bearer token")), // Velocidade
        LLMPProvider.LocalFallback("ollama://llama3.2:3b") // Offline
    )

    suspend fun processQuery(
        query: StudyQuery,
        context: LearningContext
    ): StudyResponse {
        return providers.firstNotNullOrNull { provider ->
            try {
                when (provider) {
                    is LLMPProvider.Primary -> processWithGoogleAI(query, context, provider)
                    is LLMPProvider.Secondary -> processWithGroq(query, context, provider)
                    is LLMPProvider.LocalFallback -> processWithOllama(query, context, provider)
                }
            } catch (e: Exception) {
                logProviderFailure(provider, e)
                null
            }
        } ?: StudyResponse.Error("Todos os provedores indisponíveis")
    }
}
```

3.3 Geração Inteligente de Conteúdo Educacional

Sistema de Templates Adaptativos: O assistente utiliza gabaritos *templates*) especializados para diferentes tipos de conteúdo educacional, adaptando-

se automaticamente ao contexto da disciplina, nível de dificuldade e padrões de aprendizagem individual.

Geração Contextual de Flashcards:

```
class IntelligentContentGenerator {
    suspend fun generateFlashcards(
        topic: String,
        difficulty: DifficultyLevel,
        cardType: FlashcardType,
        userContext: UserLearningProfile
    ): List<GeneratedFlashcard> {
        val prompt = buildPrompt {
            systemContext = """
                Você é um especialista em ${userContext.subjectArea} criando flashcards
                para estudantes de nível ${difficulty.name.lowercase()}.
                Considere o histórico de performance: ${userContext.recentPerformance}
                Localizações de estudo preferenciais: ${userContext.favoriteLocations}
            """

            when (cardType) {
                is FlashcardType.FrenteVerso -> {
                    userPrompt = "Crie 5 flashcards de frente e verso sobre $topic"
                    examples = listOf(
                        "Frente: O que é polimorfismo em OOP?",
                        "Verso: Capacidade de objetos de diferentes classes responderem à mesma interface de forma específica"
                    )
                }
                is FlashcardType.Cloze -> {
                    userPrompt = "Crie 3 flashcards cloze sobre $topic com lacunas estratégicas"
                    examples = listOf(
                        "O algoritmo {{c1::QuickSort}} tem complexidade média de {{c2::O(n log n)}})"
                    )
                }
                is FlashcardType.MultiplaEscolha -> {
                    userPrompt = "Gere 4 questões de múltipla escolha sobre $topic com distratores plausíveis"
                    strategy = "Distratores baseados em erros conceituais comuns"
                }
            }
        }

        return llmOrchestrator.processQuery(prompt, userContext.toLearningContext())
            .parseAsFlashcards()
            .validateAndEnrich()
    }
}
```

3.4 Validação Semântica Avançada de Respostas

Sistema de Avaliação Inteligente: O assistente implementa validação semântica sofisticada que vai além da correspondência exata, analisando a essência conceitual das respostas dos estudantes.

Implementação de Validação Semântica:

```
class SemanticResponseValidator {
    suspend fun validateResponse(
        userResponse: String,
        expectedAnswer: String,
        context: FlashcardContext
    ): ValidationResult {

        val validationPrompt = """
            Analise se a resposta do estudante demonstra compreensão correta do conceito.

            Pergunta: ${context.question}
            Resposta Esperada: $expectedAnswer
            Resposta do Estudante: $userResponse

            Considere:
            1. Correção conceitual (peso 60%)
            2. Completude da resposta (peso 25%)
            3. Clareza de expressão (peso 15%)

            Retorne JSON com:
            - score (0-100)
            - feedback personalizado
            - sugestões de melhoria
            - conceitos que precisam ser reforçados
        """

        return llmOrchestrator.processValidation(validationPrompt)
            .let { result ->
                ValidationResult(
                    score = result.score,
                    isCorrect = result.score >= 70,
                    feedback = result.feedback,
                    suggestions = result.suggestions,
                    reinforcementTopics = result.reinforcementTopics,
                    encouragement = generateEncouragement(result.score, context.userHistory)
                )
            }
    }
}
```

Sistema de Dicas Contextuais Inteligentes

Geração Adaptativa de Dicas: O sistema analisa o padrão de erros do usuário e gera dicas progressivas que guiam o aprendizado sem revelar diretamente a resposta.

Implementação de Sistema de Dicas:

```
class IntelligentHintSystem {
    suspend fun generateProgressiveHints(
        flashcard: Flashcard,
        userHistory: List[AttemptHistory],
        currentAttempt: Int
    ): HintSequence {

        val hintStrategy = analyzeUserPattern(userHistory)

        val hintsPrompt = """
            Gere uma sequência de 3 dicas progressivas para a questão:
            "${flashcard.question}"

            Padrão de erros do usuário: ${hintStrategy.errorPattern}
            Tentativa atual: $currentAttempt

            Dica 1: Orientação conceitual geral (sem revelar resposta)
            Dica 2: Direcionamento mais específico
            Dica 3: Quase revelação da resposta (usar apenas se necessário)

            Mantenha tom encorajador e educativo.
            """

        return llmOrchestrator.processQuery(hintsPrompt, flashcard.toContext())
            .parseAsHintSequence()
            .adjustForDifficulty(hintStrategy.recommendedDifficulty)
    }

    private fun analyzeUserPattern(history: List[AttemptHistory]): HintStrategy {
        val recentErrors = history.takeLast(10)

        return when {
            recentErrors.count { it.timeToAnswer < 5 } > 7 ->
                HintStrategy.TooFast("Usuário responde muito rápido, precisa de reflexão")
            recentErrors.count { it.score < 50 } > 6 ->
                HintStrategy.ConceptualGaps("Lacunas conceituais identificadas")
            recentErrors.all { it.location == recentErrors.first().location } ->
                HintStrategy.LocationBias("Distração por localização específica")
            else -> HintStrategy.Balanced("Padrão equilibrado de aprendizagem")
        }
    }
}
```

Recomendações Personalizadas de Estudo

Motor de Recomendação Baseado em IA: O sistema analisa padrões de desempenho, contexto espacial, horários de estudo e preferências para gerar recomendações altamente personalizadas.

Sistema de Recomendações:

```
class PersonalizedRecommendationEngine {
    suspend fun generateStudyRecommendations(
        user: UserProfile,
        currentLocation: Location?,
        timeOfDay: LocalTime
    ): StudyRecommendations {

        val analysisPrompt = """
            Analise o perfil de aprendizagem e gere recomendações personalizadas:

            Performance recente: ${user.recentPerformance.summarize()}
            Horário atual: $timeOfDay
            Localização: ${currentLocation?.name ?: "Não detectada"}
            Padrões temporais: ${user.studyPatterns.temporalAnalysis()}
            Localizações produtivas: ${user.favoriteLocations.rankByPerformance()}

            Recomende:
            1. Melhor tipo de flashcard para este momento
            2. Tópicos prioritários baseados em gaps identificados
            3. Duração ideal da sessão
            4. Localização recomendada (se aplicável)
            5. Estratégia de revisão espaçada otimizada
        """

        return llmOrchestrator.processRecommendation(analysisPrompt, user.toContext())
            .let { recommendations ->
                StudyRecommendations(
                    preferredCardTypes = recommendations.cardTypes,
                    priorityTopics = recommendations.topics,
                    sessionDuration = recommendations.duration,
                    recommendedLocation = recommendations.location,
                    spacingStrategy = recommendations.spacing,
                    motivationalMessage = generateMotivation(user.currentStreak,
                                                                recommendations.confidence)
                )
            }
    }
}
```

3.5 Assistente Conversacional Educacional

Interface Natural de Aprendizagem: O assistente oferece interface conversacional para dúvidas, esclarecimentos e suporte educacional contínuo, mantendo contexto de conversas anteriores e progresso acadêmico.

Implementação do Chat Educacional:

```
class EducationalChatAssistant {
    suspend fun processEducationalQuery(
        userQuestion: String,
        conversationHistory: List[ChatMessage],
        studyContext: StudyContext
    ): ChatResponse {

        val chatPrompt = buildEducationalPrompt {
            context = """
                Você é um tutor pessoal especializado em ${studyContext.currentSubject}.
                Histórico de estudo: ${studyContext.recentTopics}
                Dificuldades conhecidas: ${studyContext.strugglingTopics}
                Nível de conhecimento: ${studyContext.proficiencyLevel}
            """

            conversationContext = conversationHistory.takeLast(5)
                .joinToString("\n") { "${it.sender}: ${it.content}" }

            guidelines = """
                1. Responda de forma educativa, não apenas informativa
                2. Conecte com tópicos já estudados quando relevante
                3. Sugira flashcards ou exercícios relacionados
                4. Mantenha tom encorajador e motivacional
                5. Identifique oportunidades de revisão espaçada
            """
        }

        val response = llmOrchestrator.processChatQuery(chatPrompt)

        return ChatResponse(
            message = response.content,
            suggestedActions = extractSuggestedActions(response),
            relatedFlashcards = identifyRelatedContent(userQuestion, studyContext),
            followUpQuestions = generateFollowUp(response, studyContext)
        )
    }
}
```

4 Arquitetura Híbrida: Local, Servidor Kotlin e Nuvem

4.1 Sistema de Armazenamento Tripartite

Preservação dos Requisitos de Servidor: O projeto mantém obrigatoriamente o desenvolvimento de servidor web com API REST em Kotlin usando Ktor. Esta decisão preserva a continuidade educacional enquanto adiciona camadas modernas de sincronização em nuvem.

Arquitetura de Dados Estratificada:

1. **Camada Local:** SQLite/Room para funcionamento offline e performance
2. **Servidor Kotlin/Ktor:** API REST para compartilhamento entre usuários e backup
3. **Sincronização Firebase:** Colaboração em tempo real e recursos sociais

Implementação do Servidor Ktor:

```
fun Application.configureRouting() {
    routing {
        route("/api/flashcards") {
            get { /* Listar flashcards públicos */ }
            post { /* Criar novo flashcard */ }
            put("/{id}") { /* Atualizar flashcard */ }
            delete("/{id}") { /* Remover flashcard */ }
        }
        route("/api/statistics") {
            get("/user/{userId}") { /* Estatísticas do usuário */ }
            post("/performance") { /* Registrar desempenho */ }
        }
        route("/api/ai-services") {
            post("/validate-response") { /* Validação semântica */ }
            post("/generate-content") { /* Geração de flashcards */ }
            post("/get-hints") { /* Sistema de dicas */ }
        }
    }
}
```

4.2 Estratégia de Sincronização Inteligente

Orquestração de Múltiplas Fontes: O sistema gerencia inteligentemente a sincronização entre armazenamento local, servidor Kotlin personalizado e serviços Firebase, otimizando para diferentes cenários de uso e conectividade.

Priorização de Dados:

- **Local First:** Operações básicas funcionam offline
- **Servidor Kotlin:** Compartilhamento e descoberta de conteúdo
- **Firebase:** Recursos sociais e colaboração em tempo real
- **Resolução Inteligente:** Algoritmos de merge para conflitos de dados

5 Serviços Gratuitos para Estudantes: Recursos Essenciais para Desenvolvimento

Provedores de IA e LLM Gratuitos

1. Google AI Studio (Primeira Opção - Mais Popular)

Características:

- **Limite Gratuito:** 1 milhão de tokens por minuto com Gemini 2.5 Flash
- **Modelos Disponíveis:** Gemini Pro, Gemini Flash, Gemini Vision
- **Vantagens:** Velocidade excepcional, integração com ecossistema Google
- **Implementação:** API REST simples com documentação abrangente
- **Ideal para:** Validação de respostas, geração de conteúdo, análise semântica

```
// Configuração Google AI Studio
class GoogleAIProvider : LLMPProvider {
    private val apiKey = BuildConfig.GOOGLE_AI_API_KEY
    private val client = GoogleAIClient(apiKey)

    override suspend fun processQuery(query: String): LLMResponse {
        return client.generateContent(
            model = "gemini-2.5-flash",
            prompt = query,
            maxTokens = 1000
        )
    }
}
```

2. Groq (Segunda Opção - Velocidade Extrema)

Características:

- **Performance:** Mais de 300 tokens por segundo
- **Modelos:** Llama 3.3 70B, Mixtral, Code Llama
- **Limite:** Generoso para desenvolvimento e prototipagem
- **Ideal para:** Respostas em tempo real, validação instantânea

3. Hugging Face Inference API (Terceira Opção - Diversidade)

Características:

- **Modelos:** 300+ modelos open-source disponíveis
- **Gratuidade:** Tier gratuito robusto para experimentação
- **Flexibilidade:** Ampla gama de modelos especializados
- **Comunidade:** Maior ecossistema de modelos educacionais

4. Cerebras (Quarta Opção - Eficiência)

Características:

- **Tier Gratuito:** Substancial para projetos estudantis
- **Velocidade:** Inferência acelerada por hardware especializado
- **Modelos:** Foco em modelos otimizados para performance

5. Mistral AI (Quinta Opção - Europa)

Características:

- **Modelos:** Open-source europeus com foco em privacidade
- **API Gratuita:** Disponível para projetos educacionais
- **Codestral:** Especializado em geração de código

Plataformas de Desenvolvimento em Nuvem

5.0.1 GitHub Student Developer Pack

Benefícios Essenciais:

- **GitHub Codespaces:** 90 horas gratuitas mensais
- **GitHub Copilot:** Assistência de código com IA
- **Repositórios privados ilimitados**
- **GitHub Pages:** Hosting gratuito para documentação

```
// Configuração recomendada no devcontainer.json
{
  "name": "StudyBoost Lite Development",
  "image": "mcr.microsoft.com/devcontainers/universal:2-linux",
  "features": {
    "ghcr.io/devcontainers/features/kotlin:1": {},
    "ghcr.io/devcontainers/features/android-sdk:1": {}
  },
  "postCreateCommand": "chmod +x setup.sh && ./setup.sh",
  "extensions": [
    "mathiasfrohlich.kotlin",
    "vscjava.vscode-android"
  ]
}
```

Microsoft Azure for Students

Recursos Disponíveis:

- **\$100 em créditos anuais** sem necessidade de cartão de crédito
- **Azure Cognitive Services:** APIs de IA gratuitas
- **App Service:** Hosting para APIs backend
- **Azure Functions:** Computação serverless

Google Cloud Platform for Students

Benefícios:

- **\$300 em créditos iniciais**
- **Google Cloud Skills Boost:** Cursos gratuitos de IA
- **Vertex AI:** Plataforma de machine learning
- **Firebase:** Backend-as-a-Service completo

AWS Educate

Recursos:

- \$100 em créditos AWS
- AWS Classroom: Ambiente educacional
- Acesso a tutoriais de ML e IA
- Job Board para oportunidades

5.1 Ferramentas de Desenvolvimento Gratuitas

Cursor for Students

Características:

- 1 ano gratuito de Cursor Pro com email universitário
- IA integrada: Assistência contextual de código
- Debugging inteligente: Correção automática de erros
- Sandboxes: Ambientes de experimentação instantâneos

6 Estrutura Detalhada da Equipe e Fluxo de Trabalho

Equipe 1: Arquitetura Core e Flashcards Tradicionais

Desenvolvedor de Flashcards Tradicionais (A1)

Responsável por implementar os quatro tipos obrigatórios de flashcards (frente/verso, cloze, digite resposta, múltipla escolha) com fidelidade aos requisitos originais, expandidos com recursos modernos de UI e validação inteligente.

Tarefas Específicas:

- Implementação completa dos 4 tipos de flashcards em Jetpack Compose
- Integração com sistema de IA para validação semântica
- Interface de criação intuitiva com preview em tempo real
- Sistema de importação/exportação de flashcards

- Validação de qualidade e consistência do conteúdo

Arquiteto de Sistema GPS (A2)

Implementa o sistema de localizações favoritas com até 7 locais, integração de geofencing e algoritmos que evitam repetição excessiva de conteúdo baseada em localização, preservando o requisito central do projeto anterior.

Responsabilidades Técnicas:

- Sistema de geofencing com até 7 localizações favoritas
- Algoritmo de rotação inteligente baseado em localização
- Analytics espaciais para performance por localização
- Integração com sistema de repetição espaçada
- Interface de mapa para configuração de locais

Equipe 2: Servidor Kotlin e Integração Híbrida

Desenvolvedor Servidor Ktor (B1)

Desenvolve obrigatoriamente o servidor web com API REST em Kotlin usando Ktor, implementando endpoints para criação, compartilhamento e download de flashcards entre usuários, mantendo compatibilidade com os requisitos estabelecidos.

Implementações Essenciais:

- API REST completa em Ktor para gestão de flashcards
- Endpoints para compartilhamento público de conteúdo
- Sistema de autenticação e autorização
- Integração com serviços de IA para processamento server-side
- Backup e sincronização de dados entre dispositivos

Especialista em Sincronização Híbrida (B2)

Gerencia a orquestração entre armazenamento local, servidor Kotlin personalizado e Firebase, garantindo que estatísticas permaneçam locais enquanto permite compartilhamento de conteúdo através de múltiplos canais.

Arquitetura de Sincronização:

- Orquestração entre SQLite local, Ktor e Firebase
- Resolução inteligente de conflitos de dados

- Sincronização offline-first com queue de operações
- Cache inteligente para otimização de performance
- Monitoramento de conectividade e fallbacks

Equipe 3: Repetição Espaçada e Analytics Avançados

Engenheiro de Repetição Espaçada (C1)

Implementa algoritmo de repetição espaçada integrado com sistema GPS, garantindo que questões não sejam repetidas excessivamente na mesma localização, enquanto adiciona capacidades adaptativas baseadas em IA.

Algoritmos e Features:

- SM-2+ com adaptações baseadas em localização
- Predição de dificuldade usando IA
- Otimização de intervalos por padrões individuais
- Sistema de agendamento inteligente
- Integração com analytics temporais e espaciais

Especialista em Estatísticas Locais (C2)

Desenvolve sistema robusto de analytics armazenado localmente no dispositivo, com capacidades de exportação para o servidor Kotlin e visualizações avançadas de desempenho por localização e tipo de flashcard.

Sistema de Analytics:

- Dashboard de performance com visualizações interativas
- Métricas por localização, horário e tipo de flashcard
- Análise de padrões de aprendizagem temporal
- Exportação de dados para servidor Kotlin
- Relatórios de progresso e recomendações

Equipe 4: IA Avançada e Interface Moderna

Especialista em IA Educacional (D1)

Integra múltiplos LLMs para validação inteligente de respostas, geração de distratores para múltipla escolha, e criação de dicas contextuais para flash-cards cloze, aprimorando a experiência educacional tradicional.

Sistemas de IA Implementados:

- Orquestrador multi-LLM com fallback inteligente
- Validação semântica avançada de respostas
- Geração automática de conteúdo educacional
- Sistema de dicas progressivas contextuais
- Chat educacional com tutor IA

Designer de Interface Jetpack Compose (D2)

Cria interface gráfica de alta qualidade usando Jetpack Compose, Material Design 3 e design responsivo, garantindo que a qualidade visual seja diferencial competitivo conforme especificado no projeto anterior.

Qualidade Visual Diferencial:

- Design System completo em Material Design 3
- Animações fluidas e micro-interações
- Interface responsiva para diferentes tamanhos de tela
- Tema claro/escuro com cores dinâmicas
- Acessibilidade e usabilidade avançadas

Equipe 5: Multimídia e Garantia de Qualidade

Desenvolvedor de Recursos Multimídia (E1)

Implementa sistema de criação de conteúdo rico incluindo áudio, imagens, LaTeX e formatação avançada de texto, expandindo as capacidades dos flash-cards tradicionais sem comprometer a simplicidade.

Recursos Multimídia:

- Editor de texto rico com formatação avançada
- Sistema de gravação e reprodução de áudio

- Processamento e otimização de imagens
- Renderização de expressões matemáticas LaTeX
- Integração com armazenamento híbrido (local/nuvem)

Coordenador de Qualidade e Integração (E2)

Garante que todos os requisitos originais sejam preservados enquanto valida a integração harmoniosa de recursos modernos, coordenando testes de compatibilidade entre componentes locais, servidor Kotlin e serviços em nuvem.

Garantia de Qualidade:

- Testes automatizados para todos os requisitos mínimos
- Integração contínua com validação de qualidade
- Testes de performance e otimização
- Coordenação de integração entre equipes
- Documentação técnica e preparação de apresentação

7 Stack Tecnológico Híbrido Unificado

Tecnologias Preservadas (Requisitos Anteriores)

- **Linguagem:** Kotlin (obrigatório para cliente e servidor)
- **Framework Servidor:** Ktor para API REST
- **UI Framework:** Jetpack Compose (obrigatório)
- **Banco Local:** SQLite/Room para persistência local
- **Arquitetura:** MVVM com Android Architecture Components

Tecnologias Adicionadas (Expansão Moderna)

- **Sincronização Nuvem:** Firebase Firestore para recursos sociais
- **Injeção Dependência:** Dagger Hilt para modularidade
- **Processamento IA:** Integração com múltiplos LLMs
- **Geolocalização:** Fusion Location Provider com Geofencing
- **Armazenamento Mídia:** Combinação de armazenamento local e Firebase Storage

Configuração Recomendada do Ambiente

Stack de Desenvolvimento Otimizado

```
// build.gradle.kts (Module: app)
dependencies {
    // Core Android
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.compose.ui:ui:1.5.4")
    implementation("androidx.compose.material3:material3:1.1.2")
    implementation("androidx.navigation:navigation-compose:2.7.5")

    // Architecture Components
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    kapt("androidx.room:room-compiler:2.6.1")

    // Dependency Injection
    implementation("com.google.dagger:hilt-android:2.48.1")
    implementation("androidx.hilt:hilt-navigation-compose:1.1.0")
    kapt("com.google.dagger:hilt-compiler:2.48.1")

    // Network & API
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")

    // AI Integration (Free Tiers)
    implementation("ai.api:google-ai-client:2.8.1") // Google AI Studio
    implementation("com.huggingface:inference-api:1.2.0") // Hugging Face
    implementation("io.ktor:ktor-client-android:2.3.6") // Para Groq e outras APIs

    // Firebase (Free Tier Generoso)
    implementation(platform("com.google.firebase:firebase-bom:32.5.0"))
    implementation("com.google.firebase:firebase-auth-ktx")
    implementation("com.google.firebase:firebase-firestore-ktx")
    implementation("com.google.firebase:firebase-storage-ktx")

    // Location Services
    implementation("com.google.android.gms:play-services-location:21.0.1")
    implementation("com.google.android.gms:play-services-maps:18.2.0")

    // Multimedia
    implementation("androidx.media3:media3-exoplayer:1.2.0")
    implementation("coil-kt:coil-compose:2.5.0")
    implementation("io.github.vanpra.compose-material-dialogs:datetime:0.9.0")

    // Testing
    testImplementation("junit:junit:4.13.2")
```

```

testImplementation("androidx.room:room-testing:2.6.1")
androidTestImplementation("androidx.compose.ui:ui-test-junit4:1.5.4")
androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}

```

Configuração de Múltiplos Provedores IA

```

// ai_config.yaml (para diferentes ambientes)
development:
  primary_provider: "google_ai_studio"
  fallback_providers: ["huggingface", "groq", "local_ollama"]
production:
  primary_provider: "google_ai_studio"
  fallback_providers: ["groq", "huggingface", "azure_openai"]
limits:
  daily_requests: 1000
  cost_threshold: 5.0 # USD
  fallback_when_exceeded: true
  cache_responses: true
  cache_duration_hours: 24

```

Sistema de Gestão de Recursos

```

class ResourceManager {
  suspend fun optimizeAIUsage(
    query: AIQuery,
    userTier: UserTier
  ): AIProviderChoice {

    val usage = getCurrentUsage()

    return when {
      // Priorize provedores gratuitos sempre
      usage.googleAI < GOOGLE_DAILY_LIMIT ->
        AIProvider.GoogleAI(cost = 0.0, priority = 1)

      usage.huggingFace < HF_DAILY_LIMIT ->
        AIProvider.HuggingFace(cost = 0.0, priority = 2)

      usage.groq < GROQ_DAILY_LIMIT ->
        AIProvider.Groq(cost = 0.0, priority = 3)

      // Fallback local sempre disponível
      else ->
        AIProvider.LocalOllama(cost = 0.0, performance = Performance.REDUCED)
    }
  }
}

```

8 Cronograma Adaptado para Requisitos Duais

Semana 1: Fundação e Requisitos Obrigatórios

Dias 1-2: Implementação dos Requisitos Mínimos

Foco absoluto na implementação dos requisitos fundamentais do projeto anterior, garantindo que nenhum elemento obrigatório seja comprometido.

Tarefas Críticas:

- Criação dos quatro tipos obrigatórios de flashcards
- Sistema GPS com 7 localizações favoritas funcionando
- Configuração inicial do servidor Ktor com endpoints básicos
- Armazenamento local SQLite/Room operacional
- Interface básica em Jetpack Compose com qualidade visual
- Algoritmo de repetição espaçada básico implementado

Dias 3-4: Algoritmo de Repetição Espaçada Integrado

Expansão do sistema básico com integração GPS e primeiras capacidades de IA.

Desenvolvimentos:

- Integração completa repetição espaçada + GPS
- Sistema de estatísticas locais funcionando
- API REST básica para compartilhamento de flashcards
- Integração inicial com Google AI Studio (gratuito)
- Validação semântica básica para respostas texto

Semana 2: Aprimoramentos e Recursos Modernos

Dias 5-6: IA Avançada e Interface Premium

Implementação dos sistemas de IA avançados e aprimoramento da interface para nível diferencial.

Features Avançadas:

- Sistema multi-LLM com fallback inteligente

- Geração automática de conteúdo com IA
- Sistema de dicas progressivas contextuais
- Interface premium em Material Design 3
- Recursos multimídia (áudio, imagens, LaTeX)

Dias 7-8: Integração Completa e Sincronização

Orquestração de todos os componentes em sistema coeso e funcional.

Integrações Finais:

- Sincronização híbrida (local + Ktor + Firebase)
- Chat educacional com tutor IA
- Analytics avançados por localização
- Sistema de recomendações personalizadas
- Recursos sociais básicos (grupos, compartilhamento)

Semana 3: Validação e Apresentação

Dias 9-10: Testes Exaustivos e Validação de Requisitos

Validação metódica de todos os requisitos originais e novos recursos.

Checklist de Validação:

- Todos os 4 tipos de flashcards funcionando perfeitamente
- Sistema GPS com 7 localizações operacional
- Repetição espaçada evitando repetições por localização
- Servidor Ktor com API REST completa
- Compartilhamento de flashcards entre usuários
- Interface Jetpack Compose com qualidade diferencial
- Recursos de IA funcionando com provedores gratuitos
- Sincronização híbrida estável

Dias 11-12: Demonstração e Documentação

Preparação final para apresentação e documentação completa.

Entregáveis Finais:

- Apresentação de máximo 10 slides conforme especificado
- Demonstração ao vivo do aplicativo funcionando
- Documentação detalhada sobre decisões tomadas, papéis dos membros e principais dificuldades enfrentadas.
- Relatório completo sobre uso de LLMs com exemplos de prompts
- Análise crítica do impacto da IA na experiência educacional
- Código limpo, documentado e pronto para avaliação
- Vídeo demonstrativo curto (3-5 minutos)

9 Critérios de Avaliação Unificados

Requisitos Mínimos Obrigatórios (Peso 60%)

Funcionalidades Essenciais Preservadas:

- Flashcards Tradicionais: Quatro tipos implementados corretamente (15%)
- Sistema GPS: 7 localizações favoritas funcionando perfeitamente (10%)
- Repetição Espaçada: Algoritmo evitando repetições por localização (15%)
- Servidor Kotlin/Ktor: API REST funcional para compartilhamento (10%)
- Armazenamento: Local + servidor operacionais (5%)
- Interface Premium: Jetpack Compose com qualidade diferencial (5%)

9.1 Aprimoramentos Modernos (Peso 30%)

Recursos Avançados Implementados:

- Integração IA Multi-LLM: Sistema inteligente funcionando (8%)
- Validação Semântica: Respostas avaliadas por IA (6%)
- Recursos Multimídia: Áudio, imagens, LaTeX funcionais (5%)

- Sincronização Híbrida: Integração local + Ktor + Firebase (5%)
- Analytics Avançados: Performance por localização e padrões (3%)
- Chat Educacional: Tutor IA conversacional (3%)

9.2 Apresentação e Documentação (Peso 10%)

CrITÉRIOS de Apresentação Mantidos:

- Apresentação: Máximo 8 slides, conteúdo visível e claro (2%)
- Demonstração: Aplicativo funcionando ao vivo ou vídeo (2%)
- Documentação LLM: Relatório detalhado sobre uso de IA (2%)
- Análise Crítica: Reflexão sobre impacto educacional da IA (2%)
- Apresentação Profissional: Tom adequado e cordialidade (1%)
- Qualidade Técnica: Código limpo e documentado (1%)

10 Estratégia LLM Educacional Integrada

Preservação da Experiência de Aprendizagem

O uso extensivo de LLMs é estruturado para acelerar desenvolvimento sem comprometer o valor educacional, permitindo que estudantes foquem na compreensão de conceitos arquiteturais enquanto automatizam tarefas repetitivas.

Aplicações Recomendadas de LLM por Equipe

Equipe 1 (Flashcards + GPS):

// Prompt

Crie uma implementação Kotlin para flashcard **do** tipo cloze que:

1. Permita múltiplas lacunas em um texto
2. Valide respostas semanticamente, não apenas por string exata
3. Integre com sistema de localização para evitar repetições
4. Use Jetpack Compose com animações suaves
5. Siga padrões MVVM com Repository pattern

Equipe 2 (Servidor + Sync):

// Prompt

Implemente uma API REST em Ktor que:

1. Gerencie CRUD de flashcards com autenticação
2. Permita compartilhamento público de decks
3. Integre com múltiplos provedores de IA para processamento
4. Implemente rate limiting e cache inteligente
5. Exponha endpoints para analytics de uso

Equipe 3 (IA + Analytics):

// Prompt

Crie um sistema de repetição espaçada que:

1. Use IA para prever dificuldade inicial de flashcards
2. Adapte intervalos baseado em localização GPS do usuário
3. Analise padrões temporais de aprendizagem
4. Gere recomendações personalizadas de estudo
5. Visualize métricas em gráficos interativos

Monitoramento de Uso de IA

```
class LLMUsageTracker {
    fun trackPromptEffectiveness(
        prompt: String,
        response: String,
        userSatisfaction: Int,
        taskCompleted: Boolean
    ) {
        // Analytics para melhorar prompts ao longo do projeto
        val effectiveness = PromptEffectiveness(
            prompt = prompt.hash(),
            responseQuality = assessResponseQuality(response),
            userSatisfaction = userSatisfaction,
            taskSuccess = taskCompleted,
            timestamp = System.currentTimeMillis()
        )

        // Armazenar para análise na apresentação final
        usageDatabase.insertPromptAnalytics(effectiveness)
    }

    fun generateLLMUsageReport(): LLMReport {
        return LLMReport(
            totalPrompts = getTotalPrompts(),
            mostEffectivePrompts = getTopPrompts(10),
            providersUsed = getProviderDistribution(),
            costOptimizations = getCostSavings(),
            educationalImpact = assessLearningImpact(),
        )
    }
}
```

```

        challenges = identifiedChallenges(),
        recommendations = futureRecommendations()
    )
}
}

```

10.1 Reflexão Obrigatória sobre Ferramentas IA (Para Apresentação)

Documentação Requerida: Cada equipe deve preparar análise detalhada incluindo:

1. Ferramentas Utilizadas:

- Google AI Studio, Groq, Hugging Face, Ollama
- GitHub Copilot, Cursor, ChatGPT
- Justificativa para escolha de cada ferramenta

2. Exemplos de Prompts Mais Eficazes:

- Prompts que geraram código utilizável diretamente
- Estratégias que funcionaram vs. que falharam
- Evolução dos prompts ao longo do projeto

3. Impacto Educacional:

- Como a IA acelerou o aprendizado
- Conceitos que ainda precisaram ser estudados tradicionalmente
- Balanceamento entre eficiência e compreensão profunda

4. Desafios e Limitações:

- Problemas encontrados com diferentes provedores
- Situações onde IA atrasou ao invés de acelerar
- Estratégias para validar código gerado por IA

Parte dessas informações devem aparecer de forma resumida nos slides, com a reflexão completa no relatório.

11 Discussão final

Este projeto unificado representa uma evolução cuidadosamente planejada que preserva integralmente a essência educacional comprovada dos semestres anteriores - flashcards tradicionais, repetição espaçada, integração GPS, desenvolvimento de servidor Kotlin e interface premium - enquanto adiciona uma camada sofisticada de recursos que refletem as demandas contemporâneas do desenvolvimento mobile profissional.

11.1 Inovações adicionadas

- IA educacional que aprimora sem substituir conceitos fundamentais
- Recursos gratuitos priorizados garantindo acessibilidade financeira
- Arquitetura híbrida que demonstra versatilidade técnica
- Análise crítica de IA promovendo reflexão sobre ferramentas modernas
- Qualidade profissional preparando para mercado de trabalho

A combinação de requisitos tradicionais sólidos com recursos de IA avançados cria uma experiência de aprendizagem que é simultaneamente fundamentada na continuidade pedagógica e inovadora na preparação para o futuro do desenvolvimento mobile.

Este projeto garante que os estudantes dominem tanto os fundamentos atemporais da programação quanto as ferramentas emergentes que definirão sua carreira profissional.

12 Referências iniciais

Referências iniciais É bastante fácil encontrar tutoriais escritos e/ou em vídeo sobre os temas abordados neste trabalho.

Separei algumas referências como ponto de partida, mas podem usar as que desejarem.

Sugiro que cada membro do grupo leia ou veja tutoriais diferentes para poder compartilhar suas opiniões com o grupo.

Os grupos podem compartilhar dicas e sugestões com outros grupos usando o canal da disciplina. Por exemplo, se acharem que um tutorial é bom ou se acharem outros além dos sugeridos.

Escritos sobre repetição espaçada e recordação ativa

1. [The Science behind spaced repetition](#)

2. [Como usar a técnica de repetição espaçada para aprender novas línguas](#)
3. [Repetição Espaçada: o que é e como fazer?](#)
4. [Active recall: use a recordação ativa nos seus estudos](#)
5. [Conheça os métodos de estudo](#)
6. [O guia para impulsionar seus estudos](#)

Anki Anki é um aplicativo gratuito bastante conhecido e a seguir estão algumas informações sobre ele:

1. [Página do site](#)
2. [How to use Anki for Studying Engineering](#)
3. [CrowdAnki](#)
4. [Ankidroid](#) aplicativo para o Anki, podem usar para inspiração mas ele mais sofisticado do que o necessário para este trabalho.
5. [Intro to AnkiDroid - High Yield](#)
6. [Ankidroid wiki](#)
7. [Tipos de Notas e Cartões do Anki – Conceitos e Exemplos de Aplicações](#)

Aplicativos com Quiz

1. [Quiz App In Jetpack Compose, Clean Architecture, MVVM, Retrofit – Papaya Coders](#)
2. [Quiz App Android Studio Kotlin Project tutorial - Quiz App Kotlin Programming](#)
3. [Kotlin & Android Development Course: Build a Quiz Application](#)
4. [The Notes App \(MVVM + ROOM Database\) in Android Studio using Kotlin - CRUD](#)
5. [Flashcard App Kotlin](#)

Algoritmos de repetição espaçada em Kotlin

1. [Spaced repetition algorithm implementation in Kotlin](#)
2. [FSRS for Android \(Kotlin\)](#)

API REST em Kotlin

1. [How to create RESTful APIs in Kotlin with Ktor](#)
2. [Integrate a database with Kotlin, Ktor, and Exposed](#)
3. [Utilizando o Ktor para criar um CRUD e REST API com Kotlin](#)
4. [Creating Your Reactive REST API with Kotlin and Ktor Part I](#)
5. [Kotlin RESTful API](#)
6. [Building a REST API with Ktor and Exposed](#)
7. [Criando uma API com Ktor](#)
8. [Ktor Todo-List Backend](#)
9. [Simple web server Ktor, Sqlite, Exposed, Mustache](#)

Outros tutoriais

1. [Working with Images — Creating a Random Guessing Game App — Android App Development in Kotlin](#)